

Task 8 Individual Reflective Report

Group Project Spring 2026

Eneko ORHATEGARAY

Montrésor Group

Contents

1	Context and Task 1 Baseline	2
2	Hindsight on Personality Fit	3
3	My Contribution Across the Project	4
3.1	Technical implementation	4
3.2	Visualisation and figure generation	4
3.3	Quality assurance and analytical review	4
3.4	Communication and alignment	5
3.5	Late-stage integration support	5
4	Group Performance and Agile Review	6
4.1	What worked	6
4.2	What did not work well	6
4.3	Agile conclusion	7
5	Role Evolution: Planned vs Real	8
6	Lessons Learned and Career Transfer	9
6.1	Career transfer	9
7	Peer Assessment (0-5)	10
7.1	Scoring rubric	10
7.2	Scores table	10
7.3	Individual analyses	10
8	Conclusion	12

Chapter 1

Context and Task 1 Baseline

In Task 1 I described myself as an INFP-T "Mediator" with Evaluator and Innovator strengths. I said I would take a supporting role as a extbfTechnical Designer & Quality Analyst, focusing on code review, system design, and keeping the team working well together. I also said I was not naturally suited to highly visible leadership or front-facing coordination.

Those were my starting assumptions. This report looks back at whether they held up.

High-level summary: I did bring the analytical and quality-focused mindset I promised, but I ended up doing a lot more hands-on coding and visualisation than I expected. Task 1 got my working style roughly right, but underestimated how much technical ownership I would take on.

Chapter 2

Hindsight on Personality Fit

The INFP-T label was not wrong. I do think before I speak, I do notice when team debates are going nowhere, and I do prefer to understand the big picture before diving into details. Those traits helped me catch issues in our cooling-tower geometry for example, and in keeping discussions constructive.

But Task 1 also made it sound like I would stay in the background. In reality, when the group needed someone to build the Task 6 visual pipeline and generate the figures, I proposed myself and took ownership of the task since it seemed like an interesting and challenging task. So the "Mediator" idea that I would only contribute after careful reflection was partly true, but it did not capture that I would also jump in and do technical work directly.

One thing Task 1 got right was my weakness with documentation. I prioritised writing code and making plots over writing long report sections, which meant my contribution was sometimes clearer in the repository than in the written deliverables.

Chapter 3

My Contribution Across the Project

3.1 Technical implementation

My biggest technical contribution was in Task 6. I wrote a large part of the optimisation pipeline, including the frustum surface-area and volume calculations, the scenario definitions, and the parameter bounds that kept the cooling-tower shapes realistic. I also worked on the convergence logic and the penalty-function experiments so we could compare SA, PSO, and BFGS fairly across all eight scenarios.

3.2 Visualisation and figure generation

I spent a lot of time on the plotting scripts for Task 6. I built the code that generated the cooling-tower cross-sections, 3D renders, and convergence trajectory plots. Those figures went into the Task 6 report and were reused in Task 7. My view was that good optimisation results do not matter if nobody can see what the tower looks like or how the algorithms behaved, so I treated the visual output as a core deliverable to make it easier to understand the results.

3.3 Quality assurance and analytical review

I kept the "Evaluator" habit of checking things. I verified that our scenario definitions were mathematically consistent, that unit conversions were correct, and that the Abaqus inputs in Task 7 kept the same geometry scales as Task 6. I also sanity-checked convergence plots and flagged cases where penalty tuning gave unrealistic shapes. This was the quality-analyst role I was expected to have in Task 1, except it happened in parallel with my coding rather than as a separate review step afterwards.

3.4 Communication and alignment

I participated in meetings a decent amount and I contributed when i thought it would be beneficial: summarising technical options when discussions went in circles, asking questions about constraints to set the direction, and making sure coding decisions were written down enough for others to follow. I also acted as the link between the figures and the report text, explaining what the plots meant so the report writers could caption them accurately.

3.5 Late-stage integration support

During the final Task 7 push from 2026-03-24 to 2026-03-26, I helped validate that our figures matched the Abaqus outputs and that the report text aligned with the visual evidence. This was not my main area, but everyone was helping wherever needed at that point.

Overall, my actual contribution matched what I promised in Task 1 — analytical thinking, quality checks, creative problem-solving — but with much more coding and visualisation than I originally planned.

Chapter 4

Group Performance and Agile Review

4.1 What worked

Early tools and structure helped. We set up Notion, a Kanban board, and a Gantt chart in Task 2, and we actually used them in the first phase. That gave everyone a shared view of deadlines and stopped the early tasks from drifting.

Iteration got faster in the later tasks. By Tasks 6 and 7 we were running short cycles: generate plots, review them, fix colours or labels, regenerate. That tight loop was essential for turning raw optimisation outputs into report-quality figures.

Open task allocation kept people engaged. When tasks stopped being strictly tied to the roles we defined in Task 2, everyone felt free to work on whatever needed doing. I think that kept motivation up and prevented the bottlenecks you get when one person owns a task and they are busy, or when one person gets tired of doing the same thing over and over.

4.2 What did not work well

Ownership got fuzzy. When everyone does a bit of everything, it becomes harder to say who exactly built what. That did not hurt our delivery, but it does make organisation more difficult.

Documentation lagged behind code. Because we were all jumping between tasks, comments and handover notes were sometimes thin. I was guilty of this: my plotting scripts were clean, but I did not always explain why I chose certain viewing angles or colour scales and we didn't always have time for meetings to discuss it.

Stand-ups got shorter near deadlines. We still talked, but the formal Agile rituals became less reflective as the pressure increased. We were iterating, but without the explicit retrospectives you are supposed to have.

4.3 Agile conclusion

Our Agile process worked because it was flexible. The shift from fixed roles to open tasks was a natural evolution, and in my view it was a good one. The downside was that we lost some ownership clarity. Next time I would keep the flexibility but update the task tracker more often so everyone knows who is doing what.

Chapter 5

Role Evolution: Planned vs Real

At the start, assigning roles made sense. It reduced ambiguity and let us agree on tools and timelines quickly. I was happy with the Technical Designer & Quality Analyst role because it matched how I naturally work.

Later on, strict roles became unrealistic. The tasks were too complex and the deadlines too tight for everyone to stay in their lane. I ended up writing more core code than I expected, generating more figures than planned, and spending less time in "pure review" mode than my Task 1 self had predicted.

I think this shift was a good thing. It meant broader skill development, faster problem-solving, and a stronger sense of shared ownership. The cost was that occasionally two people started the same thing because nobody had claimed it.

My own trajectory:

- extbfPlanned: Technical Designer & Quality Analyst.
- extbfReal: Developer / visualiser, quality checker, and late-stage integrator.

That worked well for me. In future projects I would still welcome that fluidity, but I would make a habit of explicitly claiming tasks at the start of each sprint so contributions stay traceable.

Chapter 6

Lessons Learned and Career Transfer

Visual communication is an important and technical skill. Building the plotting pipeline early, with reproducible scripts, saved us a lot of time in Tasks 6 and 7.

Quality checking works better when it is built in. Catching geometry issues while I was coding them was more effective than reviewing everything at the end.

I need to make my work more visible. I assumed the output would speak for itself, but in a team project a quick update in the shared tracker or a brief note in stand-up helps people understand what you are doing.

Fluid teams need lightweight ownership signals. A simple "I will take this" comment in Notion prevents overlap without reintroducing rigid bureaucracy.

6.1 Career transfer

I am aiming for a career in data science in production or CAD design engineering. I plan to apply these lessons by:

- building reproducible visual pipelines from the start,
- embedding sanity checks in modelling scripts so quality is automatic,
- documenting design decisions in commit messages or short decision logs,
- staying flexible about roles but using task trackers to keep ownership clear.

Chapter 7

Peer Assessment (0-5)

7.1 Scoring rubric

Each member is scored on six criteria (0-5 each):

- Technical Quality (25%)
- Delivery Reliability (20%)
- Integration and Handover Quality (15%)
- Problem-Solving Under Constraints (15%)
- Collaboration Quality (15%)
- Adaptability and Learning (10%)

7.2 Scores table

Member	Technical Quality	Delivery Reliability	Integration and Handover	Problem-Solving Under Constraints	Collaboration Quality	Adaptability and Learning	Weighted score (/5)
Achille Larregle	4.6	4.6	4.7	4.3	4.5	4.3	4.53
Eneko Orhategaray	4.6	4.1	4.3	4.0	4.2	4.4	4.29
Aiert Ceccon	4.7	3.8	4.3	4.2	4.5	4.4	4.33
Paul Brocvielle	4.6	4.0	3.9	4.2	3.9	4.6	4.21

7.3 Individual analyses

extbfAchille Larregle (4.53/5)

Strengths: he kept the project organised, handled reporting and integration under deadline pressure, and made sure things got submitted.

Improvement area: delegating integration work earlier would spread the load and give others more ownership of final packaging.

Evidence: high-ownership integration in Task 6 and Task 7.

extbfAiert Ceccon (4.33/5)

Strengths: reliable technical support, good collaboration, and flexible when priorities changed. Solid day-to-day coding and meeting facilitation.

Improvement area: clearer end-to-end ownership from coding through to final report formatting would make his contribution easier to trace.

Evidence: consistent support across delivery phases, strong in early technical sprints.

extbfPaul Brocvielle (4.21/5)

Strengths: good conceptual problem-solving and analytical thinking, especially in Tasks 3 and 4. Helped us avoid weak design choices early on.

Improvement area: maintaining initiative during the high-pressure finalisation phases would help the team converge faster near deadlines.

Evidence: meaningful participation in Tasks 3-4, more variable visibility in Tasks 6-7 integration.

extbfSelf-assessment: Eneko Orhategaray (4.29/5)

Strengths: strong analytical and visual contribution in Task 6, particularly geometry, plotting, and quality checks. I combined creative problem-solving with rigorous sanity-checking.

Improvement area: I need to claim tasks more explicitly in real time and document my reasoning so my contribution trace is easier to follow.

Evidence: sustained technical and visual work during Task 6 and support in the late Task 7 integration window.

Chapter 8

Conclusion

Task 1 predicted my direction reasonably well: I contributed through analytical thinking, creative problem-solving, and quality awareness. Task 8 shows that I also took on more direct technical ownership than I expected. The project moved from fixed roles to a more fluid way of working, and I found that shift worked well for me.

My main takeaway is that my value is highest when I combine technical building with visual clarity and analytical rigour. My main improvement target is to keep building, but make the process more visible to the team through better documentation and explicit task claiming.